

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – Low Level Design (LLD)

on

Identity Security Framework for Supply Chain

Student Name: Alvia Dsouza

**Industry Mentor:
Mr.Shashank**

Guided by: Ms. Divya N Shetty

Table of Contents

Sl.no	Contents	Page
1	Introduction	1-2
1.1	Scope of the document	
1.2	Intended Audience	
1.3	System overview	
2	System Design	3-9
2.1	Application Design	
2.2	Process Flow	
2.3	Information Flow	
2.4	Components Design	
2.5	Key Design Considerations	
2.6	API Catalogue	
3	Data Design	9-10
3.1	Data Model	
3.2	Data Access Mechanism	
3.3	Data Retention Policies	
3.4	Data Migration	
4	Interfaces	11
5	State and Session Management	11
6	Caching	12
7	Non-Functional Requirements	12-13
7.1	Security Aspects	
7.2	Performance Aspects	
8	References	13

1. Introduction

This Low Level Design (LLD) document describes the internal working of the *Identity Security Framework for Supply Chain*, a web-based authentication system built using Python, Flask, Flask-Login, SQLAlchemy, and SQLite.

It covers the route logic, validation rules, database schema, password hashing mechanism, session handling, and role-based access control as implemented in the source code.

The system demonstrates two fundamental cybersecurity concepts : Authentication, which is the process of verifying a user's identity through their credentials, and Authorization, which controls what an authenticated user is permitted to access based on their assigned role. These concepts are applied in a supply chain context involving three user roles: Admin, Supplier, and Distributor.

1.1 Scope of the document

This document covers the implementation-level design of the following: the User data model defined in `models.py` including column definitions, data types, constraints, and the `UserMixin` interface; the Flask application configuration in `app.py` covering secret key setup, database URI, SQLAlchemy initialization, and Flask-Login configuration; all seven route handlers including their HTTP methods, input extraction logic, validation sequences, database interactions, session operations, and response behavior; the password hashing and verification mechanism using Werkzeug; the session creation, persistence, and termination lifecycle managed by Flask-Login; the role-based authorization checks applied to the admin and delete routes; and the structure of all HTML templates including form fields, Jinja2 template variables, flash message rendering, and role-based UI elements.

This document does not cover deployment configuration, server infrastructure, network setup, or future feature additions such as email verification, password reset, or API-based access.

1.2 Intended Audience

This document is intended for developers, technical evaluators, and academic reviewers who need a precise understanding of how every part of the system is implemented. It assumes the reader has working knowledge of Python, the Flask web framework, relational databases, HTTP request-response cycles, and basic web security concepts such as password hashing and session management.

1.3 System overview

The Identity Security Framework for Supply Chain is a multi-page Flask web application that provides secure user registration, credential-based login, session-managed access to protected pages, and role-based access control across three user types. The application is structured around a single SQLite database table accessed through Flask-SQLAlchemy, and uses Flask-Login to manage user sessions across requests.

The application has seven routes: the root route which redirects to login, the signup route, the login route, the dashboard route, the logout route, the admin route, and the delete route. Each route enforces security at every step unauthenticated users cannot access protected pages, non-admin users cannot access admin pages, and no password is ever stored or transmitted in plain text at any point in the system.

The technology stack consists of Python 3.x as the programming language, Flask as the WSGI web framework, Flask-Login for session management, Flask-SQLAlchemy as the ORM layer, SQLite as the relational database, Werkzeug for password hashing, and Jinja2 as the HTML templating engine. The frontend is built with plain HTML and CSS with no external JavaScript frameworks.

2. System Design

2.1 Application Design

The application is organized into two Python files and a templates folder:

app.py — Main application file containing the Flask app instance, all configuration settings, database initialization, Flask-Login setup, and all route handler functions.

models.py — Contains the SQLAlchemy database instance and the User class which maps to the users table in SQLite.

templates/ — Contains four HTML files: login.html, signup.html, dashboard.html, and admin.html, all rendered using the Jinja2 templating engine.

instance/database.db — The SQLite database file auto-generated on first application startup via db.create_all().

User Model: The User class inherits from UserMixin (Flask-Login) and db.Model (SQLAlchemy). UserMixin provides the is_authenticated, is_active, is_anonymous, and get_id() properties required by Flask-Login.

App Configuration:

- SECRET_KEY : Used by Flask to sign session cookies securely.
- SQLALCHEMY_DATABASE_URI : Set to sqlite:///database.db.
- SQLALCHEMY_TRACK_MODIFICATIONS : Set to False to suppress warnings.
- login_manager.login_view : Set to 'login' to redirect unauthenticated users.

project/

├── app.py

├── models.py

├── templates/

| ├── login.html

| ├── signup.html

| ├── dashboard.html

| └── admin.html

└── instance/

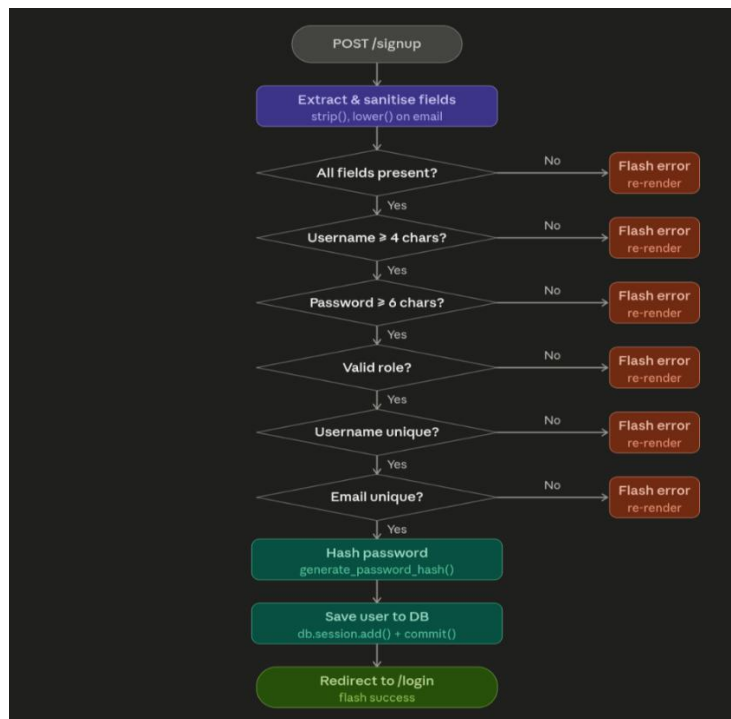
└── database.db

2.2 Process Flow

Signup Route (/signup — GET & POST)

On GET, renders signup.html. If `current_user.is_authenticated` is `True`, redirects to dashboard instead.

On POST, executes the following steps: checks all six fields are present → username must be at least 4 characters → password must be at least 6 characters → role must be one of ('admin', 'supplier', 'distributor') → username must not already exist in DB → email must not already exist in DB → hashes password using `generate_password_hash()` → creates new User object, stages with `db.session.add()` and writes with `db.session.commit()` → flashes success message and redirects to /login.



Login Route (/login — GET & POST)

On GET, renders login.html. If authenticated, redirects to dashboard.

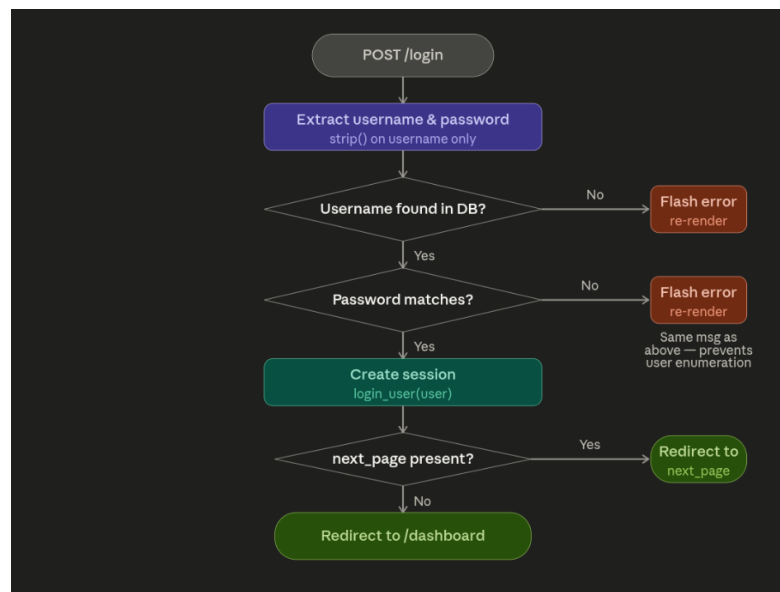
On POST: extracts username (stripped) and password (not stripped) → queries DB with `User.query.filter_by(username=username).first()` → verifies password with `check_password_hash()` → if valid, calls `login_user()` to store user ID in session cookie → checks `request.args.get('next')` and redirects there if present, otherwise redirects to /dashboard → if credentials fail, flashes a single generic error 'Invalid username or password.' to prevent username enumeration.

Logout Route (/logout): `@login_required`. Calls `logout_user()`, flashes 'You have been logged out.', redirects to /login.

Dashboard (/dashboard): `@login_required`. Renders dashboard.html passing `current_user` (first_name, last_name, email, role).

Admin (/admin): @login_required. Checks `current_user.role == 'admin'`, returns "Access Denied" if not. Otherwise fetches `User.query.all()` and renders `admin.html` with the full user list.

Delete (/delete/int:id): @login_required. Checks admin role. Prevents self-deletion by comparing target user ID with `current_user.id`. If valid, deletes user with `db.session.delete()` and `db.session.commit()`, flashes success, redirects to /admin.



2.3 Information Flow

The user interacts with an HTML form → browser sends an HTTP request → Flask routes it to the correct handler → the handler extracts form data, validates, and performs ORM queries → SQLAlchemy reads/writes the SQLite database → the handler calls `render_template()` or `redirect()` → flash messages are consumed by the next template → browser renders the response.

2.4 Components Design

The system is built around five components.

The Registration Component accepts form inputs from the signup form, runs a 7-step sequential validation sequence checking field presence, username length, password length, role validity, and uniqueness of both username and email, hashes the password using Werkzeug's `generate_password_hash()`, and writes the new User object to the database using `db.session.add()` and `db.session.commit()`.

The Authentication Component queries the database using `User.query.filter_by(username=username).first()` and calls `check_password_hash()` to verify the submitted password against the stored hash in a timing-safe manner. On success it invokes `login_user()` to create a signed session cookie.

The Session Guard Component is implemented as the `@login_required` decorator from Flask-Login. It intercepts every unauthenticated request to a protected route and redirects to `/login`, preserving the originally requested URL in the next query parameter so the user can be returned there after login.

The Role-Based Authorization Component performs a manual `current_user.role != 'admin'` check inside the `/admin` and `/delete/<id>` route handlers after the `@login_required` check passes, implementing a two-layer access control model first verify authentication, then verify role.

The Admin User Management Component provides the admin panel and delete functionality. The admin route calls `User.query.all()` to retrieve all registered users and renders `admin.html`. The delete route fetches the target user by ID, compares it against `current_user.id` to prevent self-deletion, then calls `db.session.delete(user)` and `db.session.commit()` to permanently remove the record.

2.5 Key Design Considerations

Password hashing uses PBKDF2-HMAC-SHA256 with a random salt via Werkzeug's `generate_password_hash()`, ensuring no two stored hashes are identical even for identical passwords. The login route returns the same error message whether the username is unknown or the password is wrong, preventing username enumeration attacks.

Flask session cookies are HMAC-signed using the `SECRET_KEY` - they are readable but cannot be forged. All string inputs are stripped of whitespace, and email addresses are lowercased to prevent duplicate registrations with different capitalization. Username and email uniqueness is enforced at both the application level (pre-insert query) and the database level (UNIQUE constraint), providing dual-layer protection. The delete route compares the target user's ID against `current_user.id` before proceeding, preventing self-deletion.

2.6 API Catalogue

Route	Method	Auth Required	Role	Description
/	GET	No	None	Redirects to /login
/signup	GET, POST	No	None	User registration
/login	GET, POST	No	None	User authentication
/dashboard	GET	Yes	Any	User dashboard
/logout	GET	Yes	Any	Clears session

/admin	GET	Yes	Admin	View all users
/delete/<id>	GET	Yes	Admin	Delete a user by id

3. Data Design

3.1 Data Model

The system uses a single entity, User, which stores all registration and authentication data. The entity diagram below represents the structure of the users table in the SQLite database.

USERS		
int	id	PK
varchar	first_name	
varchar	last_name	
varchar	email	UK
varchar	username	UK
varchar	password	
varchar	role	

The password field stores the Werkzeug PBKDF2 hash string and never contains plain text. The role field determines the user's access level within the system and can hold one of three values: admin, supplier, or distributor.

3.2 Data Access Mechanism

All database operations are performed through Flask-SQLAlchemy. No raw SQL is written anywhere in the application.

- **Fetch user by username:** `User.query.filter_by(username=username).first()` used during login and duplicate checks.
- **Fetch user by ID:** `db.session.get(User, int(user_id))` used in the `user_loader` callback.
- **Fetch all users:** `User.query.all()` used in the admin panel.
- **Insert user:** `db.session.add(new_user)` then `db.session.commit()`.
- **Delete user:** `db.session.delete(user)` then `db.session.commit()`

3.3 Data Retention Policies

User records are stored permanently in the local SQLite database file until explicitly deleted by an admin user through the `/delete/<id>` route. There is no automated expiry, soft delete, or archival mechanism in the current version. Deleted records are permanently removed from the database with no recovery mechanism. The `database.db` file itself persists on the filesystem for the lifetime of the application installation. There is no mechanism for users to delete their own accounts in the current version only an admin can remove user records.

3.4 Data Migration

The database schema is created automatically on first startup using `db.create_all()`. No formal migration strategy is implemented. If the User model is changed in a future version, the database file would need to be recreated or a tool like Flask-Migrate would need to be introduced to handle schema changes without data loss.

4. Interfaces

signup.html — Registration form with fields for first name, last name, email, username, password, and role selection via three styled radio buttons (Admin, Supplier, Distributor). Flash messages displayed above the form.

login.html — Login form with username and password fields. Flash messages displayed on invalid login. Link to signup page for new users.

dashboard.html — Displays the logged-in user's name, email, and role. Contains a logout link and an admin panel link visible only to admin users.

admin.html — Table listing all registered users with their ID, username, email, and role. Each row has a delete button. Back button navigates to dashboard.

All templates use Jinja2 for dynamic rendering and Flask's `get_flashed_messages()` for displaying success and error messages.

5. State and Session Management

Flask-Login manages user sessions through signed cookies.

- On `login_user(user)`, the user's ID is stored in a session cookie signed with the app's `SECRET_KEY`.
- On every subsequent request, the `user_loader` callback retrieves the full User object from the database using the ID stored in the cookie, making it available as `current_user`.
- Routes protected with `@login_required` check `current_user.is_authenticated` before allowing access.
- On `logout_user()`, the session is cleared and the cookie is invalidated.
- The cookie is signed using HMAC and cannot be forged or modified by the client.

6. Caching

The current version of the application does not implement any caching mechanism. Every HTTP request results in fresh database queries being executed. The user object is reloaded from the database on every request via the `user_loader` callback, and the admin panel executes `User.query.all()` on every page load. For the purposes of this project a single-user local development environment this is entirely acceptable and adds no measurable overhead.

In a production environment with concurrent users, introducing query-level caching using Flask-Caching with a memory or Redis backend would significantly reduce database load, particularly for the admin panel which reads all user records.

7. Non-Functional Requirements

7.1 Security Aspects:

- **Password Hashing:** Passwords are hashed using PBKDF2-HMAC-SHA256 with a random salt via Werkzeug. Plain text passwords are never stored or logged.
- **Session Security:** Session cookies are signed with the `SECRET_KEY` using HMAC. Tampered cookies are rejected by Flask automatically.
- **Authentication:** All sensitive routes are protected with `@login_required`. Unauthenticated users are redirected to login.
- **Authorization:** Admin routes additionally check `current_user.role`. Non-admin users are denied access server-side.
- **Input Validation:** All inputs are validated server-side. Fields are stripped of whitespace and emails are lowercased before storage.
- **Self-Deletion Guard:** An admin cannot delete their own account, preventing accidental logout.

7.2 Performance Aspects:

- SQLite is suitable for the single-user, local development scope of this project.
- The UNIQUE constraints on username and email create indexes automatically, keeping login and duplicate-check queries efficient.
- Flask's built-in development server is single-threaded and used for testing purposes only. A WSGI server like Gunicorn would be needed for production.
- User.query.all() in the admin route retrieves all records at once — for a larger user base, pagination would be recommended.

8. References

- Flask Documentation: <https://flask.palletsprojects.com>
- Flask-Login Documentation: <https://flask-login.readthedocs.io>
- FlaskSQLAlchemy Documentation: <https://flasksqlalchemy.palletsprojects.com>
- Werkzeug Documentation: <https://werkzeug.palletsprojects.com>
- SQLite Documentation: <https://www.sqlite.org/docs.html>
- Python 3 Documentation: <https://docs.python.org/3/>
- OWASPAuthenticationCheatSheet: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html